Generics in Java

=> Generics :-

➔ Generics means parametrized types which means that we can provide any type of parameter to the classes, interfaces or methods.
➔ Generics were introduced in JDK 1.5 version.
➔ Generics are represented by angular braces - <>

➔ The main objective of Generics are :-
1. To provide type safety
2. To resolve type casting problem

➔ By default "arrays are type safe"
➔ Now for collections, till JDK 1.4 version, collections were not generic types.
➔ In JDK 1.5 version, Generic Collections were introduced.

➔ NOTE : We can only provide non-primitive values in generics

---

=> Generic Classes :-

➔ If any class is declared with type parameters then it is known as Generic Class
➔ Generic classes can be user-defined classes or predefined classes (collections classes)

➔ Generic type can be any valid identifier name

➔ We can provide any number of parameters in generics

=> Generic Bounded Types :-

➔ We can bound the type parameter for a particular range by usin extends keyword. And this concept is known as Generic Bounded Type Concept.

➔ Syntax : class A<T extends X>

(X can be any class or interface)

➔ Cases :-

1. We can only use extends keyword, not implements keyword.
2. We can only use Non-Primitive data types.
3. class A<T extends X & Y>

=> Generic Methods & Generic Wildcards (?) :-

- The wildcard (represented by the question mark ?) stands for an "unknown type".
- It is used to provide flexibility in your code, particularly when defining method parameters that need to handle different generic types.

➢ **Types of Wildcards**

1. Upper Bounded Wildcards (? extends T)

- ✓ Restricts the unknown type to be a specific type or its subtypes.
- ✓ Example: List<? extends Number> can accept List<Integer>, List<Double>, or List<Number>.
- ✓ Use Case: Ideal for "in" variables where you only need to read data from a structure.

2. Lower Bounded Wildcards (? super T):

- ✓ Restricts the unknown type to be a specific type or its supertypes.
- ✓ Example: List<? super Integer> can accept List<Integer>, List<Number>, or List<Object>.
- ✓ Use Case: Ideal for "out" variables where you are putting data into a structure.

3. Unbounded Wildcards (?):

- ✓ Represents a completely unknown type (essentially ? extends Object).
- ✓ Example: List<?> can accept a list of any type.
- ✓ Use Case: Useful when a method's logic only depends on methods from the Object class (like equals()) or when the code doesn't depend on the type parameter (like List.size()).